

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 7: PLC INTERFACING WITH MICROCONTROLLER AND PC OVER

ETHERNET/IP

SECTION 1

GROUP 9

INSTRUCTOR: DR WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV.

DR ZULKIFLI BIN ZAINAL ABIDIN

NO	NAME	MATRIC NO
1	NOR EFFENDI BIN ANUAR @ ANWAR	2310519
2	NUR AINA SAFFIYA BINTI MOHD TARMIZI	2411644
3	NUR ATIKAH BINTI KAMARUL BAHARIN	2319846

DATE OF SUBMISSION: 29 APRIL 2026

TABLE OF CONTENTS

1.0 INTRODUCTION.....	3
2.0 MATERIALS AND EQUIPMENT.....	3
3.0 EXPERIMENTAL SETUP.....	4
3.1 Hardware Setup.....	4
3.2 OpenPLC Editor Setup.....	5
4.0 METHODOLOGY.....	8
4.1.1 Circuit Setup.....	8
4.1.2 Program Development and Uploading.....	8
4.1.3 Program Operation.....	9
5.0 DATA COLLECTION.....	10
6.0 DATA ANALYSIS.....	11
7.0 RESULTS.....	12
8.0 DISCUSSION.....	13
9.0 CONCLUSION.....	14
10.0 RECOMMENDATIONS.....	14
11.0 REFERENCES.....	15
ACKNOWLEDGEMENT.....	16
STUDENTS' DECLARATION.....	16

1.0 INTRODUCTION

A Programmable Logic Controller (PLC) is a specialized industrial computer designed to automate electromechanical processes in manufacturing and machinery. Unlike standard microcontrollers, PLCs are engineered for high reliability in harsh environments and typically use Ladder Diagram (LD) language, which mimics physical relay logic circuits.

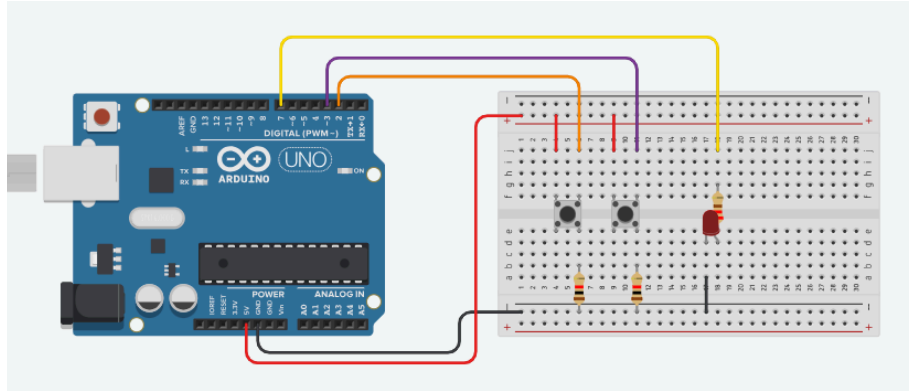
This experiment utilizes the OpenPLC Editor, an open-source, IEC 61131-3 compliant platform that allows users to write PLC code and upload it directly to supported hardware like the Arduino. By interfacing OpenPLC with an Arduino Uno, we can simulate industrial control systems—such as a Start-Stop Control Circuit—using standard consumer electronics, effectively bridging the gap between hobbyist microcontrollers and industrial automation standards.

2.0 MATERIALS AND EQUIPMENT

- OpenPLC Editor software
- Arduino Board
- 2 Push Button Switches
- LED
- Resistors
- Jumper Wires
- Breadboard

3.0 EXPERIMENTAL SETUP

3.1 Hardware Setup

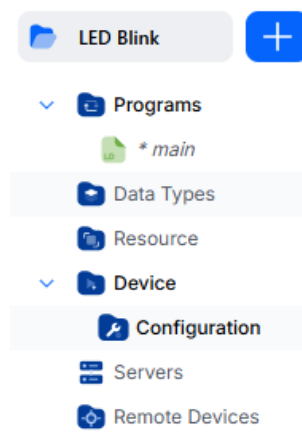


1. Connection between Arduino Uno to the Breadboard:
 - a. Connect the 5V pin from the Arduino to the positive terminal on the breadboard.
 - b. Connect the GND pin from the Arduino to the negative terminal on the breadboard.
2. Connection between Button 1 to the Arduino Uno:
 - a. Connect the top left pin of the Button 1 to the positive terminal of the breadboard.
 - b. Connect the top right pin of the Button 1 to D2 pin on the Arduino Uno.
 - c. Connect the bottom right of the Button 1 to one of the 1k Ω resistor legs.
 - d. Connect the other 1k Ω resistor leg to the negative terminal on the breadboard.
3. Connection between Button 2 to the Arduino Uno:
 - a. Connect the top left pin of the Button 2 to the positive terminal of the breadboard.

- b. Connect the top right pin of the Button 2 to D3 pin on the Arduino Uno.
 - c. Connect the bottom right of the Button 2 to one of the 1k Ω resistor legs.
 - d. Connect the other 1k Ω resistor leg to the negative terminal on the breadboard.
4. Connection between LED to the Arduino Uno:
- a. Connect the LED anode leg to one of the 220 Ω resistor legs.
 - b. Connect the other leg of the 220 Ω resistor to the D7 pin on Arduino Uno.
 - c. Connect the LED cathode leg to the negative terminal on the breadboard.

3.2 OpenPLC Editor Setup

1. Configure the OpenPLC program to Arduino Uno board:
 - a. Go to “Device” and then to “Configuration”.



- b. At the Board Setting, untick the “Compile Only”, choose “Arduino Uno (1.8.7)” at the Device section and choose the Communication Port based on the port that connect to the Arduino.

Board Settings

Compile Only ☐

Device Arduino Uno [1.8.7]

Communication Port

Specs

CPU: ATmega 328P at 16MHz

RAM: 2 KB

Flash: 32 KB

Digital Pins: 14

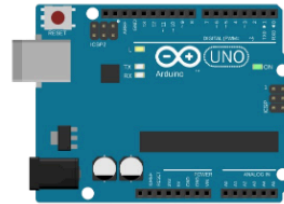
Analog Pins: 6

PWM Pins: 6

WiFi: No

Bluetooth: No

Ethernet: No



- c. At Pin Mapping, choose your output and input pins to be used on your Arduino. Make sure the address is matched to which pins choose to be used (Can refer to the internet on what address name is for the pins on Arduino).

Pin Mapping



Pin	Type	Address	Name
2	Digital Input	%IX0.0	
3	Digital Input	%IX0.1	
7	Digital Output	%QX0.0	

- d. On the Communication section, tick the “Enable Modbus RTU”, leave the “Interface” and “Baud Rate” to default choices and write 1 at the “Slave ID”.

Communication

☒ Enable Modbus RTU

Interface Serial

Baud Rate 115200

Slave ID 1

☐ RS485 EN Pin

☐ Enable Modbus TCP

2. Add all the variables needed for the task at the variable table.

Description :

Class Filter :

All

+

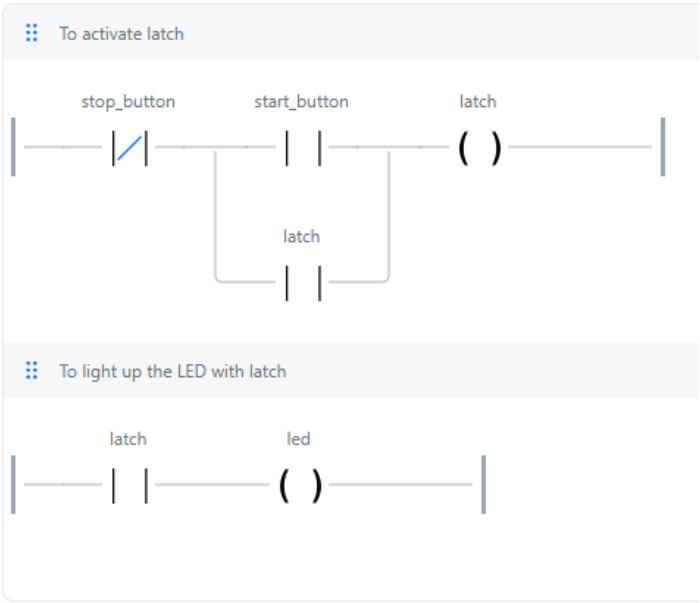
−

↑

↓

#	Name	Class	Type	Location	Initial Value	Documentation	Debug
0	start_button	Local	BOOL	%IX0.0		start button	
1	stop_button	Local	BOOL	%IX0.1		off button	
2	led	Local	BOOL	%QX0.0		led output	
3	latch	Local	BOOL			lock power	

3. Setup the rung and put the coil and contact needed for the task on the rung.



4.0 METHODOLOGY

4.1.1 Circuit Setup

- Connect the +5V and GND pins from the Arduino Uno to the respective power rails of the breadboard.
- Place two push-button switches (PB1 for Start, PB2 for Stop) on the breadboard.
- Connect PB1 to Arduino Pin 2 and PB2 to Arduino Pin 3.
- Install 10k Ω pull-down resistors (R1 and R2) for each switch to ensure stable logic states.
- Connect the anode of a red LED to Arduino Pin 7 through a 560 Ω current-limiting resistor (R3).
- Connect the cathode of the LED to the GND rail.

4.1.2 Program Development and Uploading

- Start a new project in the OpenPLC Editor and select LD (Ladder Diagram) as the programming language.
- Define the following variables in the software table:

UP_Start: Class: Local, Type: BOOL, Location: %IX0.0.

DOWN_Stop: Class: Local, Type: BOOL, Location: %IX0.1.

LED: Class: Local, Type: BOOL, Location: %QX0.0.

Latch: Class: Local, Type: BOOL (internal memory bit).

- Develop the rung logic, incorporating a negated contact for the Stop button and a latch contact in parallel with the Start button to maintain the state.
- Add a second rung where the Latch variable controls the LED coil.

- Click the compile icon to verify the code.
- Connect the Arduino to the PC, select the correct Board Type and COM Port, then click Transfer to PLC.

4.1.3 Program Operation

- Pressing PB1 (UP_Start) completes the circuit in the ladder logic, energizing the Latch bit and turning the LED ON.
- Because of the parallel latch contact, the LED remains ON even after the PB1 button is released.
- Pressing PB2 (DOWN_Stop) opens the normally closed (negated) contact in the program, breaking the latch and turning the LED OFF.

5.0 DATA COLLECTION

Operation	Start Button (PB1) State	Stop Button (PB2) State	LED Status
Initial State: System powered on; no buttons pressed.	0 (LOW)	0 (LOW)	OFF
Start Initiated: Pressing and holding the Start button (PB1).	1 (HIGH)	0 (LOW)	ON
Latching State: Start button (PB1) released; system latched.	0 (LOW)	0 (LOW)	REMAINS ON
Stop Initiated: Pressing and holding the Stop button (PB2).	0 (LOW)	1 (HIGH)	OFF
Final State: Stop button (PB2) released; system reset.	0 (LOW)	0 (LOW)	REMAINS OFF

6.0 DATA ANALYSIS

Analyzing the collected data demonstrates a direct correlation between the programmed Ladder Diagram (LD) simulation logic and the real-time physical performance of the Arduino Uno microcontroller. The behavior of the system under initial conditions confirms that the $1k\Omega$ pull-down resistors effectively mitigated electrical noise, ensuring that unpressed buttons provided a true 0V logic LOW signal to the Arduino pins. The transition of the LED to an active state upon pressing PB1 verifies that the normally open contact assigned to the Start button successfully closed, creating a complete path of logic continuity that energized the internal Latch coil.

The core mechanism under analysis is the self-latching behavior observed when PB1 was released. Even though the Start button input dropped back to 0, the parallel alignment of the Latch contact provided an alternative, self-sustaining path for the virtual current to flow. This configuration successfully locked the system's state in place without requiring continuous manual input. The subsequent interruption of the circuit by PB2 validates the use of a negated (normally closed) contact within the software. When the Stop button was pressed, the negated contact flipped to an open state, instantly breaking the parallel logic path, de-energizing the Latch bit, and dropping the LED output back to a logic 0. This proves that the OpenPLC configuration, Modbus RTU communication mapping, and physical wiring operated seamlessly as an integrated, reliable consumer-grade substitute for industrial relay control logic.

7.0 RESULTS

In these experiments, we built the circuit on a breadboard using our Arduino Uno, two push buttons, two 1k ohm pull-down resistors, and a red LED with a 220 ohm resistor. After uploading the program through the correct COM port, the real hardware worked exactly like the simulation. When we pressed the start button, the red LED turned on and stayed on. The LED only turned off when we pressed the stop button.

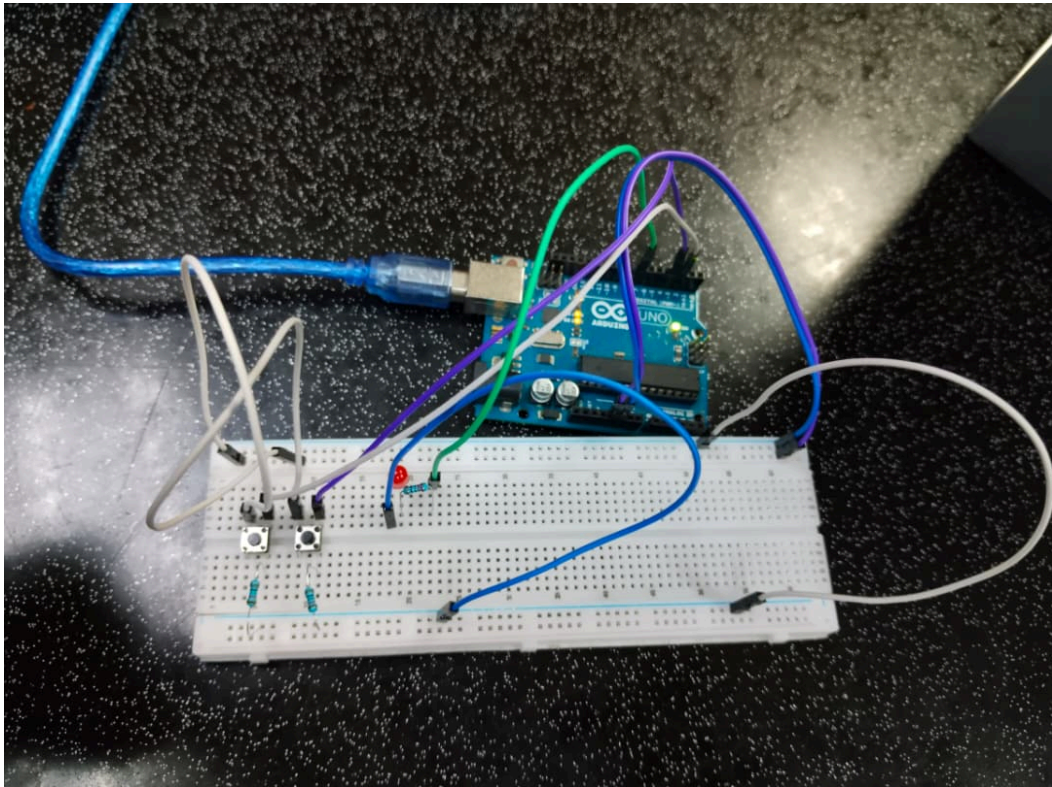


Diagram 7.1.1 show the circuit connection with the Arduino Uno

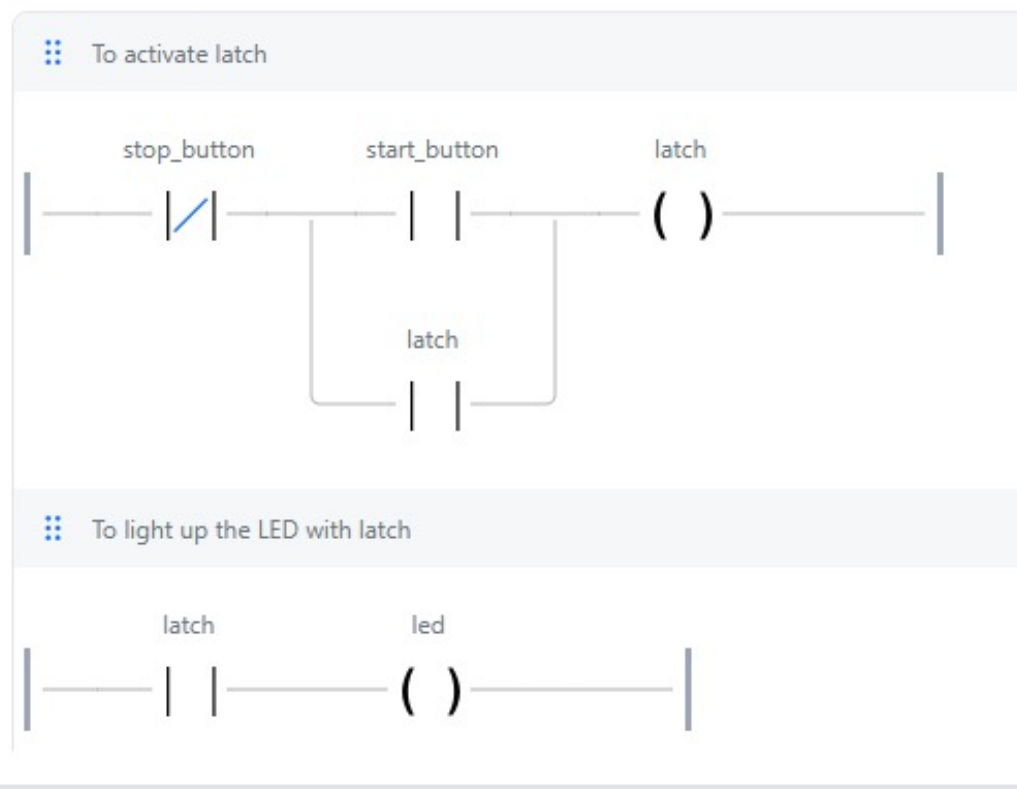


Diagram 7.1.2 show the ladder diagram of this experiments

8.0 DISCUSSION

In these experiments, we can learn how industrial automation works just using an Arduino Uno and OpenPLC software. The software shows how the self latching works. When we press UP-start, it activates the latch coil. Since we put a latch contact in parallel with the start button, the electric current finds another path to flow. This keeps the circuit on even after we let go of the start button. To stop it, we used a negated contact for DOWN-stop. Pressing the stop button opens this contact, breaks the circuit, and turns OFF the LED. As for the hardware, we use a 1K ohm connected to the push button to make sure the Arduino pins read 0V when the button is not pressed. There are some errors that often happen when conducting

these experiments. That is the connection of the circuit. Besides, the address of the pins in the software must be the same with the connection of the Arduino. Otherwise the circuit will not run.

9.0 CONCLUSION

To conclude, the experiments successfully teach us how to design and make a simulation using the OpenPLC software. The results prove that we can run industrial control programs using free open-source software and basic microcontrollers. These experiments also explain why industrial automation uses the latching logic in their system and machine. For example, a quick button press can lock a machine's state until someone presses stop.

10.0 RECOMMENDATIONS

The recommendation for this experiment is that we can use more LED indicators to show the users the status of a machine. For example, a green LED indicates the machine is safe or fully stopped. Other than that, we also can use a normally closed switch to make a safer system. The switch button should be physically wired as a normally closed switch. This way, if a wire accidentally cuts or breaks, the machine will stop automatically for safety.

11.0 REFERENCES

Logic, A. (n.d.). *Autonomy | Industrial Automation Platform*. <https://autonomylogic.com/>

Control.com, “PLC Ladder Logic on an Arduino: Introduction to OpenPLC.” [Online].

Available:

<https://control.com/technical-articles/plc-ladder-logic-on-an-arduino-introduction-to-openplc/>

Control.com, “PLC Ladder Logic on an Arduino: Building a Start-Stop Circuit.” [Online].

Available:

<https://control.com/technical-articles/plc-ladder-logic-on-an-arduino-building-a-start-stop-circuit/>

ACKNOWLEDGEMENT

We would like to sincerely express our gratitudes to our instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal and the lab technician, for the help given during the task. The knowledge and suggestions are able to guide us throughout the progress of the task. We also would like to thank our classmates for their support and help which really give positive impacts to the progression of our task.

STUDENTS' DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed

unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: 

Read [/]

Name: NOR EFFENDI BIN ANUAR @ ANWAR

Understand [/]

Matric Number: 2310519

Agree [/]

Signature: 

Read [/]

Name: NUR AINA SAFFIYA BINTI MOHD TARMIZI

Understand [/]

Matric Number: 2411644

Agree [/]

Signature: 

Read [/]

Name: NUR ATIKAH BINTI KAMARUL BAHARIN

Understand [/]

Matric Number: 2319846

Agree [/]